CAHIER DES CHARGES ET PROMPT DE CONCEPTION D'UN ERP ÉDUCATIF PROFESSIONNEL

1. Rôle et mission

Agis comme un architecte logiciel senior, expert en systèmes d'information éducatifs (SIS), en ERP SaaS multi-tenant, en ingénierie des bases de données, en Django, en conception UX/UI et en modélisation des processus académiques.

Ta mission consiste à concevoir et développer un ERP éducatif professionnel, évolutif, sécurisé, modulaire et réellement adapté aux établissements scolaires classiques et aux universités.

Le système doit prendre en charge les réalités administratives, pédagogiques et académiques de ces deux types d'établissements sans imposer une architecture artificiellement complexe.

Il ne s'agit pas simplement de créer un CRUD pour chaque entité. Il faut comprendre les relations entre les différentes opérations et concevoir une véritable logique métier cohérente.

PRINCIPE FONDAMENTAL : L'UTILISATEUR FINAL EST LA PRIORITÉ.

L'objectif n'est pas de multiplier les fonctionnalités, les champs ou les formulaires, mais de permettre aux utilisateurs d'accomplir leurs tâches rapidement, correctement et avec le minimum d'efforts.

---

2. Philosophie de conception : User-Centered Design

Le système doit être conçu autour des besoins réels des utilisateurs.

Respecter impérativement les principes suivants :

- Réduire au maximum les champs à remplir manuellement.
- Générer automatiquement les codes, références, dates et identifiants lorsque cela est possible.
- Préremplir les informations déjà connues du système.
- Éviter de demander plusieurs fois la même information.
- Ne pas multiplier les interfaces pour effectuer une même opération.
- Adapter les interfaces aux rôles et responsabilités des utilisateurs.
- Privilégier les listes déroulantes intelligentes, la recherche instantanée et les valeurs par défaut pertinentes.

- Utiliser des formulaires courts et compréhensibles.
- Regrouper les informations complémentaires dans des sections facultatives ou progressives.
- Prévoir une excellente expérience sur ordinateur, tablette et téléphone.
- Éviter les tableaux et formulaires qui imposent des défilements horizontaux excessifs sur mobile.
- Prévoir des états de chargement, des confirmations et des messages d'erreur compréhensibles.

Ne jamais confondre richesse du modèle de données et complexité de l'interface utilisateur.

Un modèle Django peut comporter plusieurs champs sans que tous soient exposés dans un formulaire.

Chaque champ affiché doit avoir une justification métier.

---

3. Architecture académique adaptable

L'ERP doit pouvoir gérer deux grandes catégories d'établissements.

A. Établissements scolaires classiques

Exemple : une école qui organise les études de la 7e année fondamentale à la philosophie.

L'établissement possède plusieurs niveaux :

- 7e année fondamentale ;
- 8e année fondamentale ;
- 9e année fondamentale ;
- NS1 ;
- NS2 ;
- NS3 ;
- NS4 / Philosophie.

Chaque niveau peut être subdivisé en plusieurs classes :

- 7e A ;
- 7e B ;
- 8e A ;
- 8e B ;
- 9e A ;
- NS1 A ;
- NS1 B.

Il faut impérativement distinguer :

Le niveau : position de l'apprenant dans le parcours scolaire.

La classe : groupe pédagogique auquel l'apprenant est affecté.

Une classe appartient à un niveau. Plusieurs classes peuvent appartenir au même niveau.

B. Établissements universitaires

Une université peut être organisée en plusieurs facultés.

Chaque faculté peut proposer plusieurs programmes d'études, options ou départements.

Ces appellations peuvent varier selon les établissements. Le système doit donc permettre de configurer le vocabulaire institutionnel utilisé sans créer artificiellement plusieurs entités pour désigner la même notion.

Exemple :

- Faculté des Sciences ;
- Faculté des Sciences Humaines ;
- Faculté des Sciences Économiques.

Chaque faculté peut proposer différents programmes d'études.

L'apprenant universitaire est inscrit dans un programme, une option ou un département selon la terminologie de l'établissement.

Le modèle doit également pouvoir gérer les années d'études, les niveaux ou les promotions universitaires si l'organisation de l'établissement les utilise.

Ne pas imposer aux universités la structure des classes des écoles classiques.

Exigence

Concevoir une architecture commune lorsque les concepts sont réellement partagés et des structures spécifiques lorsque les réalités métier sont différentes.

Éviter les champs génériques mal définis et les relations artificielles.

---

4. Gestion des années académiques

L'année académique est une entité fondamentale du système.

Elle doit être créée et configurée avant toute inscription.

Exemple : 2026-2027.

Une année académique possède notamment :

- Une désignation ;
- Une date de début ;
- Une date de fin ;
- Un statut ;
- Une configuration des périodes académiques.

Une seule année académique peut être active à la fois pour un établissement, sauf si une autre organisation est explicitement prévue.

Aucune nouvelle inscription ne peut être effectuée sans année académique active.

L'année académique doit pouvoir être organisée selon le type d'établissement :

Pour une école classique :

- 3 trimestres ;
- 4 trimestres ;
- ou toute autre organisation autorisée.

Pour une université :

- 2 semestres ;
- ou des sessions selon l'organisation universitaire.

Le système ne doit pas imposer un nombre fixe de périodes.

Les périodes doivent être configurables par établissement et par année académique.

Chaque période possède :

- Un nom ;
- Une date de début ;
- Une date de fin ;
- Un ordre ;
- Un statut.

Le système doit également gérer l'ouverture et la fermeture des périodes, notamment pour les opérations de saisie et de publication des résultats.

---

5. Gestion des apprenants et des inscriptions

Il faut impérativement distinguer le dossier permanent de l'apprenant de ses inscriptions académiques.

Dossier de l'apprenant

L'apprenant est enregistré une seule fois dans le système.

Son dossier contient ses informations personnelles, ses coordonnées, ses responsables légaux et les autres informations nécessaires.

Il possède un identifiant permanent.

Processus d'inscription

L'inscription est une opération liée à une année académique.

Pour une école classique, elle concerne :

- L'apprenant ;
- L'année académique ;
- Le niveau demandé ;
- La classe, lorsqu'elle est déterminée.

Pour une université, elle concerne :

- L'apprenant ;
- L'année académique ;
- La faculté ;
- Le programme, l'option ou le département ;
- Le niveau ou la promotion, lorsque cela est applicable.

Une nouvelle inscription doit avoir automatiquement le statut EN_ATTENTE.

Un utilisateur disposant des autorisations nécessaires doit pouvoir :

- Valider l'inscription ;
- Rejeter l'inscription ;
- Demander des corrections ou des pièces complémentaires.

Le système doit conserver la trace de la décision, de son auteur, de sa date et de son éventuel motif.

Une inscription en attente ou rejetée ne doit pas être considérée comme une inscription académique active.

Ne pas créer un module Admission indépendant obligatoire lorsque le processus d'admission peut être intégré directement à la gestion des inscriptions.

Une demande d'inscription peut donc représenter simultanément une demande d'intégration et un processus de validation administrative.

Réinscription

Un ancien apprenant ne doit jamais être recréé à chaque nouvelle année académique.

Il conserve son dossier permanent.

Le système crée simplement une nouvelle inscription pour la nouvelle année.

Il faut également prendre en charge :

- Le passage au niveau supérieur ;
- Le redoublement ;
- Le changement de classe ;
- Le transfert ;
- L'interruption du parcours ;
- La reprise des études.

Prévenir les inscriptions en double pour un même apprenant et une même année académique, sauf cas métier explicitement autorisé.

---

6. Gestion des matières, cours et barèmes

C'est un élément central de l'architecture pédagogique.

Les matières ou cours ne doivent pas être définis uniquement comme des éléments globaux de l'établissement.

Leur association doit respecter le contexte académique.

Règles métier

Un niveau peut posséder plusieurs matières.

Un autre niveau peut avoir une liste de matières totalement différente.

Une même matière peut être enseignée dans plusieurs niveaux.

Cependant, les barèmes peuvent être différents selon les niveaux.

Exemple :

Niveau	Matière	Barème
7e année	Mathématiques	100
8e année	Mathématiques	100
9e année	Mathématiques	200
NS1	Mathématiques	200
NS2	Mathématiques	300

Ces valeurs sont illustratives.

Le système doit permettre de définir les matières et leurs paramètres pédagogiques dans le contexte approprié.

Dans une école classique, plusieurs classes appartenant au même niveau doivent normalement partager les mêmes matières et les mêmes barèmes.

Il ne faut donc pas obliger l'administration à configurer séparément les matières et les barèmes pour 7e A et 7e B.

Les classes héritent de la configuration pédagogique de leur niveau, sauf dérogation explicitement autorisée.

Pour les universités, la configuration doit être rattachée au programme, au niveau d'études ou à la structure pédagogique pertinente.

Prévoir également l'affectation des enseignants aux matières, cours, niveaux, classes ou groupes concernés.

---

7. Système de notation configurable

Le système de notation doit être entièrement configurable.

Il ne faut pas imposer une notation unique de 10, 20 ou 100 points.

Exemples :

- Notation sur 10 ;
- Notation sur 20 ;
- Notation sur 100 ;
- Notation sur 200.

Chaque matière ou cours possède un barème défini dans son contexte pédagogique.

Le système doit normaliser les résultats afin de permettre les comparaisons et les calculs nécessaires.

Règle de réussite

Par défaut, un apprenant réussit lorsque sa note est supérieure ou égale à la moitié du barème applicable.

Exemples :

- Sur 10 : réussite à partir de 5 ;
- Sur 20 : réussite à partir de 10 ;
- Sur 100 : réussite à partir de 50.

Cette règle doit pouvoir être configurée au niveau institutionnel lorsque les règlements pédagogiques l'exigent.

Ne jamais coder en dur la notation sur 20.

---

8. Gestion flexible des évaluations

L'enseignant doit disposer d'une liberté pédagogique dans l'organisation de ses évaluations.

Le système ne doit pas lui imposer une structure identique pour toutes les périodes.

Exemple concret :

Pour les mathématiques en première année :

Première période :

- Interrogation : 10 points ;
- Devoir : 20 points ;
- Examen : 70 points.

Total : 100 points.

Deuxième période :

- Examen sur table : 100 points.

Total : 100 points.

Le système doit permettre ces deux organisations sans nécessiter de modification du modèle de données.

Règles de validation

1. L'enseignant choisit les composantes de son évaluation.
2. Il définit les intitulés et les barèmes de chaque composante.
3. Le système calcule automatiquement la somme des barèmes.
4. La somme doit être exactement égale au barème défini pour la matière et la période.
5. Si la somme est différente, le système doit empêcher la validation de la configuration et expliquer clairement le problème.
6. Une fois la configuration validée, l'enseignant peut saisir les notes des apprenants.
7. Le système calcule automatiquement les totaux, les moyennes et les résultats selon les règles applicables.

La configuration des évaluations doit être propre à la matière, à la classe ou au groupe concerné et à la période académique.

Une modification après saisie des notes doit être contrôlée et tracée.

Prévoir les situations d'absence, de dispense, de note non renseignée et de résultats incomplets sans les confondre avec une note égale à zéro.

---

9. Consultation et analyse des résultats

L'enseignant doit pouvoir consulter et filtrer les résultats de ses apprenants selon différents critères.

Exemples de filtres :

- Année académique ;
- Période ;
- Niveau ;
- Classe ;
- Matière ou cours ;
- Composante d'évaluation ;
- Statut de réussite ;
- Apprenant ;
- Résultats publiés ou non publiés.

Le système doit proposer des tableaux lisibles et des indicateurs utiles.

Fonctionnalités souhaitées :

- Consultation individuelle des notes ;
- Résultats collectifs ;
- Moyennes ;
- Classement, si autorisé par l'établissement ;
- Taux de réussite ;
- Identification des apprenants en difficulté ;
- Historique des résultats ;
- Exportation PDF et Excel, selon les permissions.

Les enseignants ne doivent consulter que les données relevant de leurs affectations et autorisations.

Les résultats doivent respecter les règles de confidentialité.

---

10. Gestion des rôles et permissions

Le système doit être organisé autour des responsabilités réelles.

Prévoir notamment :

- Administrateur de la plateforme ;
- Administrateur de l'établissement ;
- Directeur ;
- Secrétaire ;
- Responsable pédagogique ;
- Enseignant ;
- Comptable ;
- Apprenant ;
- Parent ou responsable légal.

Les autorisations doivent être granulaires.

Exemples :

- Un enseignant peut saisir les notes de ses classes autorisées.
- Un enseignant ne peut pas modifier arbitrairement le barème institutionnel.
- Un secrétaire peut enregistrer une demande d'inscription.
- Seul un utilisateur autorisé peut valider ou rejeter cette inscription.
- Un directeur peut consulter les statistiques globales selon ses permissions.
- Un apprenant ne peut consulter que ses propres informations.

Prévoir un historique des opérations sensibles.

---

11. Architecture technique et qualité du code

Utiliser les technologies suivantes, sauf justification technique sérieuse :

- Python ;
- Django 4.2 LTS ;
- MySQL avec encodage utf8mb4 ;
- Bootstrap 5 ;
- JavaScript lorsque nécessaire ;
- python-decouple ou django-environ pour les variables d'environnement.

Organiser le projet de manière modulaire, avec une séparation claire des responsabilités.

Les noms de modèles, commentaires, messages et interfaces doivent être en français lorsque cela est pertinent.

Respecter les bonnes pratiques Django :

- Modèles cohérents et normalisés ;
- Relations explicites ;
- Contraintes d'intégrité ;
- Transactions atomiques pour les opérations sensibles ;
- Validation métier côté serveur ;
- Permissions vérifiées côté serveur ;
- Requêtes optimisées ;
- Protection contre les doublons ;
- Tests automatisés ;
- Gestion correcte des migrations.

Utiliser les modèles abstraits uniquement pour les propriétés réellement communes.

Ne pas ajouter systématiquement les champs code, libellé et actif à tous les modèles.

Distinguer les référentiels, les entités permanentes et les opérations transactionnelles.

Préférer les statuts explicites et les choix contrôlés lorsque plusieurs états sont possibles.

Pour les opérations complexes, utiliser des services métier plutôt que de concentrer toute la logique dans les vues ou les modèles.

---

12. Architecture SaaS multi-établissements

Le système doit permettre à plusieurs établissements de travailler indépendamment sur une même plateforme.

Chaque établissement doit disposer de ses propres :

- Années académiques ;
- Structures académiques ;
- Apprenants ;
- Inscriptions ;
- Matières et cours ;
- Enseignants ;
- Classes et programmes ;
- Évaluations ;
- Résultats.

Aucune donnée d'un établissement ne doit être accessible à un autre sans autorisation explicite.

La séparation des données doit être garantie côté serveur, et non uniquement par les filtres de l'interface.

Éviter toute fuite de données entre tenants.

---

13. Exigences UX/UI

Créer une interface moderne, professionnelle, responsive et intuitive.

Privilégier :

- Une navigation claire ;
- Un tableau de bord adapté au rôle ;
- Des formulaires courts ;
- Des tableaux avec recherche, filtres et pagination ;
- Des actions accessibles rapidement ;
- Des confirmations avant les opérations sensibles ;
- Des messages d'erreur explicites ;
- Une présentation cohérente sur tous les modules.

Ne pas multiplier les boutons et les menus pour donner artificiellement une impression de richesse fonctionnelle.

Chaque écran doit correspondre à une tâche réelle.

---

14. Méthodologie de développement obligatoire

Ne commence pas immédiatement à générer des dizaines de modèles ou des milliers de lignes de code.

Procède méthodiquement.

Étape 1 : Analyse métier

Analyse les exigences et identifie :

- Les entités ;
- Les relations ;
- Les processus ;
- Les contraintes ;
- Les règles de validation ;
- Les différences entre établissements scolaires et universitaires.

Signale les ambiguïtés et les éventuelles contradictions avant de prendre une décision architecturale.

Étape 2 : Modélisation

Produis :

- Le diagramme conceptuel de données ;
- Le diagramme relationnel ;
- Les principales cardinalités ;
- Les règles d'intégrité ;
- Les flux de fonctionnement.

Justifie les choix de modélisation.

Étape 3 : Architecture applicative

Définis les applications Django, les modèles, les services, les vues, les formulaires, les permissions et les responsabilités de chaque module.

Évite la duplication fonctionnelle.

Étape 4 : Conception UX

Décris les parcours utilisateurs avant de développer les formulaires.

Pour chaque formulaire, distingue :

- Les champs obligatoires ;
- Les champs facultatifs ;
- Les champs préremplis ;
- Les champs calculés ;
- Les champs invisibles à l'utilisateur.

Étape 5 : Développement progressif

Développe un module à la fois.

Chaque module doit être fonctionnel, testé et cohérent avec les précédents.

Ne jamais modifier arbitrairement les modèles déjà établis sans analyser les conséquences sur les données existantes.

Étape 6 : Tests

Tester les scénarios métier réels, les permissions, les contraintes, les erreurs, les cas limites et l'isolation des établissements.

---

15. Résultat attendu

Je veux un ERP éducatif qui respecte les réalités du terrain et qui puisse être utilisé quotidiennement par des secrétaires, enseignants, directeurs, comptables, parents et apprenants.

Je ne veux pas :

- Des formulaires interminables ;
- Des champs répétitifs sans utilité ;
- Des modules qui reproduisent les mêmes opérations ;
- Une architecture rigide ;
- Des règles pédagogiques codées en dur ;
- Des inscriptions qui recréent les apprenants ;
- Des classes confondues avec les niveaux ;
- Des matières globales qui ignorent les barèmes spécifiques ;
- Des évaluations imposées aux enseignants ;
- Des fonctionnalités ajoutées uniquement pour complexifier le logiciel.

Je veux :

- Une logique métier solide ;
- Une base de données cohérente ;
- Une expérience utilisateur fluide ;
- Une architecture évolutive ;

- Une gestion académique flexible ;
- Une automatisation intelligente ;
- Une sécurité rigoureuse ;
- Une excellente maintenabilité.

RÈGLE FINALE : Avant d'ajouter une fonctionnalité, un champ ou un module, demande-toi toujours s'il apporte une réelle valeur à l'établissement et à l'utilisateur final. Si le système peut accomplir automatiquement une opération sans intervention humaine inutile, privilégie l'automatisation.

Commence exclusivement par l'analyse métier et la modélisation conceptuelle. Présente tes propositions, leurs justifications et les éventuels points à clarifier. Attends la validation de l'architecture avant de passer au développement.

16. DIRECTIVES DE CONCEPTION UX/UI ET IDENTITÉ VISUELLE

16.1. Philosophie générale

Ne génère pas une interface administrative générique ou un tableau de bord préfabriqué.

Je ne demande pas une interface standardisée dont les composants seraient simplement assemblés sans réflexion sur les usages.

Je veux que tu conçoives l'interface en fonction des tâches réelles, des rôles et des habitudes des utilisateurs.

Inspire-toi des principes d'expérience utilisateur des applications modernes telles que TikTok, Spotify et LinkedIn, notamment en matière de navigation mobile, de simplicité, d'accessibilité et de hiérarchisation des fonctionnalités.

Il ne s'agit pas de copier leur identité graphique, mais de reprendre les bonnes pratiques de navigation et d'expérience utilisateur.

16.2. Navigation mobile : exigence obligatoire

Sur les écrans mobiles, prévoir une barre de navigation fixe en bas de l'écran.

Cette barre doit contenir exactement cinq accès :

- Quatre liens directs vers les fonctionnalités les plus importantes du rôle connecté.
- Un cinquième accès nommé « Plus », représenté par une icône composée de trois barres horizontales.

Les quatre liens principaux doivent être dynamiques et dépendre du profil de l'utilisateur.

Exemples :

Enseignant :
Accueil, Classes, Notes, Résultats, Plus.

Secrétaire :
Accueil, Apprenants, Inscriptions, Documents, Plus.

Directeur :
Accueil, Personnel, Académique, Rapports, Plus.

Comptable :
Accueil, Paiements, Dépenses, Rapports, Plus.

Parent :
Accueil, Enfants, Résultats, Paiements, Plus.

Apprenant :
Accueil, Cours, Résultats, Profil, Plus.

Ces exemples sont indicatifs. Les liens doivent correspondre aux permissions réellement accordées à chaque utilisateur.

Fonctionnement du bouton Plus

Au clic sur « Plus », ouvrir un panneau latéral coulissant depuis le côté de l'écran.

Ce panneau doit :

- Présenter les autres fonctionnalités disponibles.
- Regrouper les fonctionnalités par catégories.
- Afficher des icônes compréhensibles.
- Respecter les permissions de l'utilisateur.
- Permettre une fermeture simple.
- Ne pas masquer définitivement la navigation principale.
- Prévoir une fermeture par bouton, touche Échap lorsque pertinente et clic à l'extérieur.

Le panneau doit être utilisable sur les petits écrans sans créer de débordement horizontal.

La navigation inférieure doit rester accessible pendant les déplacements dans l'application, sauf lorsqu'une interaction nécessite temporairement de la masquer.

Prévoir les espaces nécessaires pour éviter que la navigation fixe masque le contenu.

Respecter les zones de sécurité des appareils mobiles.

16.3. Navigation sur ordinateur

Sur les écrans de bureau, utiliser une navigation latérale structurée et une barre supérieure adaptée.

La navigation doit être organisée selon les responsabilités et les fonctionnalités.

Prévoir :

- Une sidebar claire et personnalisable ;
- Un logo et le nom de l'établissement ;
- Des menus regroupés par catégories ;
- Un état actif visible ;
- Une possibilité de réduire la sidebar ;
- Une barre supérieure avec les notifications, le profil et les actions contextuelles.

Ne pas reproduire automatiquement la navigation mobile sur ordinateur.

L'interface doit exploiter intelligemment l'espace disponible.

16.4. Adaptation par rôle

Ne pas présenter les mêmes tableaux de bord et les mêmes menus à tous les utilisateurs.

Chaque rôle doit disposer d'une interface adaptée à ses responsabilités.

Les fonctionnalités doivent être déterminées par les permissions et les affectations réelles, pas uniquement par le nom du rôle.

Un enseignant doit accéder rapidement à ses classes, ses matières et ses évaluations.

Un secrétaire doit pouvoir enregistrer et traiter rapidement les inscriptions.

Un comptable doit pouvoir accéder aux opérations financières.

Un parent doit pouvoir suivre les informations de ses enfants.

16.5. Identité visuelle

Utiliser Bootstrap 5 comme fondation technique et un système de CSS personnalisé pour construire une identité graphique propre à l'ERP.

Ne pas utiliser les styles Bootstrap par défaut sans personnalisation.

Définir des variables CSS centralisées pour :

- Les couleurs principales ;
- Les couleurs secondaires ;
- Les couleurs d'état ;
- Les espacements ;
- Les rayons des bordures ;
- Les ombres ;
- Les tailles typographiques ;
- Les points de rupture responsive.

Privilégier une interface claire, moderne, professionnelle et légère.

Éviter les combinaisons de couleurs excessivement sombres, les dégradés décoratifs inutiles, les animations envahissantes et les cartes répétées sans justification.

16.6. Responsive design

Concevoir l'interface selon une approche mobile-first.

Tester notamment :

- Les petits téléphones ;
- Les téléphones de taille standard ;
- Les grands téléphones ;
- Les tablettes ;
- Les ordinateurs portables ;
- Les écrans de bureau.

Les formulaires doivent être faciles à remplir sur téléphone.

Les tableaux doivent proposer une adaptation pertinente : colonnes prioritaires, consultation détaillée, filtres ou défilement horizontal contrôlé lorsque nécessaire.

Ne pas réduire simplement une interface desktop pour la faire entrer dans un écran mobile.

16.7. Accessibilité et ergonomie

Prévoir :

- Des zones tactiles suffisamment grandes ;
- Des contrastes lisibles ;
- Des libellés explicites ;
- Des icônes accompagnées de textes lorsque leur signification pourrait être ambiguë ;
- Une navigation clavier ;

- Des messages d'erreur compréhensibles ;
- Une hiérarchie visuelle cohérente.

Ne pas dépendre uniquement des couleurs pour communiquer les statuts.

16.8. Architecture CSS

Utiliser Bootstrap 5 pour les fondations et les composants standards.

Utiliser du CSS personnalisé organisé par composants et responsabilités.

Éviter :

- Les styles inline excessifs ;
- Les règles CSS globales qui provoquent des conflits ;
- Les répétitions de déclarations ;
- Les surcharges Bootstrap désordonnées ;
- Les feuilles de style monolithiques.

Centraliser les éléments réutilisables dans un système de design.

16.9. Méthode de réalisation

Avant de développer les interfaces, produire :

1. Une cartographie des parcours utilisateurs.
2. Une architecture de navigation par rôle.
3. Une description des écrans desktop et mobile.
4. Une description des composants réutilisables.
5. Une définition de l'identité visuelle.
6. Une stratégie responsive.

Présenter ces propositions avant leur implémentation.

Ne pas générer immédiatement des interfaces sans avoir compris les besoins des utilisateurs.

L'objectif est de construire une interface originale, cohérente et agréable à utiliser, et non de reproduire un template administratif existant.